EXPERIMENT 2

Aim: Create a Blockchain using Python

Theory:
Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using create_block() with a proof and previous hash.

2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).

3. **Hashing** – SHA-256 algorithm ensures data immutability.

4. **Validation** – is_chain_valid() checks that every block correctly references the previous hash and satisfies PoW conditions.

5. **Flask Web API** – Routes /mine_block, /get_chain, and /is_valid allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Program and Output:

Program:

```
import datetime
import hashlib
import json
```

```
class Blockchain:
    def __init__(self):
        self.chain = []
        self.transactions = []
        self.create_block(proof=1, previous_hash='0')

    def create_block(self, proof, previous_hash, transactions=None):
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash,
            'transactions': transactions if transactions is not None else []
        }

        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def create_Transactions(self, sender, receiver, amount):
        transaction = {
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        }

        self.transactions.append(transaction)

        return transaction

    def proof_of_work(self, previous_proof):
        new_proof = 1
```

```
    while True:
        hash_operation = hashlib.sha256(
            str(new_proof**2 - previous_proof**2).encode()
        ).hexdigest()

        if hash_operation[:3] == '000':
            return new_proof

        new_proof += 1

def hash(self, block):
    encoded_block = json.dumps(
        block,
        sort_keys=True
    ).encode()

    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1

    while block_index < len(chain):
        block = chain[block_index]

        # Check previous hash
        if block['previous_hash'] != self.hash(previous_block):
            return False

        # Check Proof of Work
        previous_proof = previous_block['proof']
        proof = block['proof']
```

```
            hash_operation = hashlib.sha256(
                str(proof**2 - previous_proof**2).encode()
            ).hexdigest()

            if hash_operation[:3] != '000':
                return False

            previous_block = block
            block_index += 1

        return True

# Creating a Blockchain
blockchain = Blockchain()

print(blockchain.chain)
```

Output:

[{'index': 1, 'timestamp': '2026-08-30 15:28:43.662032', 'proof': 1, 'previous_hash': '0', 'transactions': []}]

Program:

```
# Mining a new block
def mine_block():

    # Check whether transactions are available
    if len(blockchain.transactions) == 0:
        return {
            'message': 'No transactions available. Block cannot be mined.'
        }

    previous_block = blockchain.get_previous_block()

    previous_proof = previous_block['proof']

    proof = blockchain.proof_of_work(previous_proof)
```

```
    previous_hash = blockchain.hash(previous_block)

    # Store current transactions in the block
    transactions = blockchain.transactions.copy()

    # Remove transactions before mining the block
    blockchain.transactions.clear()

    block = blockchain.create_block(
        proof,
        previous_hash,
        transactions
    )

    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions']
    }

    return response
sender = input("Enter sender: ")
receiver = input("Enter receiver: ")
amount = input("Enter amount: ")

transaction = blockchain.create_Transactions(
    sender,
    receiver,
    amount
)
```

```
print("Transaction added successfully:")
print(transaction)
```

Output:

```
Enter sender: janhavi
Enter receiver: bob
Enter amount: 40000
Transaction added successfully:
{'sender': 'janhavi', 'receiver': 'bob', 'amount': '40000'}
```

Program:

```
def get_chain():
    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain),
        'pending_transactions': blockchain.transactions
    }

    return response

def is_valid():

    valid = blockchain.is_chain_valid(blockchain.chain)

    if valid:
        response = {
            'message': 'All good. The Blockchain is valid.'
        }
    else:
        response = {
            'message': 'Houston, we have a problem. The Blockchain is not valid.'
        }

    return response
```

```
print("Functions Menu:")
print("================")
print("1. Add a transaction")
print("2. Mine a block")
print("3. Display the chain")
print("4. Check the validity of the chain")

choice = int(input("Enter your choice: "))

if choice == 1:
    sender = input("Enter sender: ")
    receiver = input("Enter receiver: ")
    amount = input("Enter amount: ")

    print(
        blockchain.create_Transactions(
            sender,
            receiver,
            amount
        )
    )

elif choice == 2:

    print(mine_block())

elif choice == 3:

    print(get_chain())

elif choice == 4:

    print(is_valid())
```

```
else:

    print("Invalid choice")
```

Output:

```
Functions Menu:
================
1. Add a transaction
2. Mine a block
3. Display the chain
4. Check the validity of the chain
Enter your choice: 3


{'chain': [{'index': 1, 'timestamp': '2026-08-30 15:28:43.662032', 'proof': 1, 'previous_hash': '0',
'transactions': []}, {'index': 2, 'timestamp': '2026-08-30 15:29:49.789595', 'proof': 533,
'previous_hash': 'a779f0fec2265dbd9497ba53771b68cc911847b9b9415dbf19e07f77a2f56300',
'transactions': [{'sender': 'janhavi', 'receiver': 'bob', 'amount': '40000'}]}], 'length': 2,
'pending_transactions': []


Functions Menu:
================
1. Add a transaction
2. Mine a block
3. Display the chain
4. Check the validity of the chain
Enter your choice: 4
{'message': 'All good. The Blockchain is valid.'}
```

Output (Postman):


```
{
    "message": "Transaction added successfully.",
    "transaction": {
        "amount": 40000,
        "receiver": "siu",
        "sender": "janhavi"
    }
}
```



```
{
    "message": "Transaction added successfully.",
    "transaction": {
        "amount": 40000,
        "receiver": "siu",
        "sender": "janhavi"
    }
}
```



```
{
    "index": 2,
    "message": "Congratulations, you just mined a block!",
    "previous_hash": "80f7f57b4eb6dc9033f4b9102dd5f97f8afb7624f9b397ac80ce0510b03219f9",
    "proof": 533,
    "timestamp": "2026-08-30 21:31:34.900695",
    "transactions": [
        {
            "amount": 40000,
            "receiver": "riyu",
            "sender": "janhavi"
        }
    ]
}
```

```
{
    "chain": [
        {
            "index": 1,
            "previous_hash": "0",
            "proof": 1,
            "timestamp": "2026-08-30 21:19:26.794974",
            "transactions": []
        },
        {
            "index": 2,
            "previous_hash": "80f7f57b4eb6dc9033f4b9102dd5f97f8afb7624f9b397ac80ce0510b03219f9",
            "proof": 533,
            "timestamp": "2026-08-30 21:31:34.900695",
            "transactions": [
                {
                    "amount": 40000,
                    "receiver": "riyu",
                    "sender": "janhavi"
                }
            ]
        }
    ],
    "length": 2,
    "pending_transactions": []
}
```

```
{
    "message": "All good. The Blockchain is valid."
}
```


Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.